

Exercices — Algèbre linéaire avancée

Thèmes : déterminant, rang, élimination de Gauss, valeurs propres, vecteurs propres, SVD, PCA et compression d'images.

B. Déterminant et rang

Exercice B1 — Déterminant d'une matrice 2×2 (Débutant)

Calculer à la main le déterminant, puis vérifier avec NumPy.

4	7
2	5

Écrire la formule utilisée et détailler le calcul.

Exercice B2 — Déterminant d'une matrice 3×3 par Sarrus (Débutant)

Calculer à la main le déterminant par la règle de Sarrus, puis vérifier avec `np.linalg.det`.

2	1	3
0	-1	4
5	2	1

Exercice B3 — Rang par échelonnement de Gauss (Intermédiaire)

La troisième ligne est une combinaison linéaire des deux premières. Réduire la matrice à la main et déterminer son rang.

1	2	3
2	-1	1
5	3	7

Trouver d'abord la relation entre les lignes, puis effectuer les opérations élémentaires.

Exercice B4 — Élimination de Gauss sans NumPy (Avancé)

Implémenter `gauss_elimination(A)` avec uniquement des listes Python et des boucles. Tester sur :

2	1	-1
-4	-2	2
1	2	3

Gérer au minimum l'échange de lignes et l'élimination. Documenter les étapes.

Exercice B5 — Déterminant récursif (Avancé)

Écrire une fonction récursive calculant le déterminant d'une matrice $n \times n$ par développement selon la première ligne. Tester jusqu'à 5×5 et comparer avec `numpy.linalg.det`.

3	2
---	---

1	4	
1	2	3
0	4	5
1	0	6

Tester ensuite sur des matrices aléatoires 4×4 et 5×5 et mesurer le temps d'exécution.

C. Valeurs propres et vecteurs propres

Exercice C1 — Matrice diagonale (Débutant)

Donner directement les valeurs propres et un vecteur propre associé à chacune, en utilisant la propriété des matrices diagonales.

4	0
0	-2

Exercice C2 — Polynôme caractéristique (Intermédiaire)

Calculer à la main les valeurs propres, puis les vecteurs propres associés.

3	1
2	2

Étapes : calculer $\det(A - \lambda I)$, résoudre le polynôme caractéristique, puis résoudre $(A - \lambda I)v = 0$ pour chaque λ . Vérifier avec `np.linalg.eig`.

Exercice C3 — Power Iteration (Avancé)

Implémenter la puissance itérée sans `numpy.linalg.eig` afin d'approximer la plus grande valeur propre et son vecteur propre associé.

6	2	0	0
2	5	1	0
0	1	4	1
0	0	1	3

Initialiser $v = [1, 1, 1, 1]$. Faire au moins 50 itérations ou arrêter lorsque la variation devient inférieure à 10^{-8} . Comparer avec NumPy.

D. SVD et PCA

Exercice D1 — Première composante principale (Débutant)

Expliquer avec vos mots ce que représente la première composante principale sur ce nuage de points 2D. Identifier intuitivement la direction de variance maximale et faire un schéma.

1	1
2	2
3	2
4	4
5	4
6	5
7	6
8	6

Exercice D2 — SVD d'une matrice 3×2 (Intermédiaire)

Calculer la SVD avec `numpy.linalg.svd`, identifier U , Σ et V^T , puis vérifier $A \approx U\Sigma V^T$.

3	1
1	3
2	2

Indiquer les dimensions de U , Σ et V^T et expliquer le rôle des valeurs singulières.

Exercice D3 — PCA from-scratch (Avancé)

Construire une PCA : centrer les données, calculer la covariance, calculer et trier valeurs/vecteurs propres, puis projeter sur les deux premières composantes. Comparer avec `sklearn.decomposition.PCA(n_components=2)`.

2.1	1.9	0.5	1.0
2.5	2.4	0.7	1.2
3.0	2.8	1.0	1.5
3.4	3.2	1.1	1.7
4.0	3.7	1.5	2.0
4.5	4.1	1.8	2.3
5.0	4.6	2.0	2.6
5.4	5.0	2.2	2.8
6.0	5.5	2.5	3.1
6.4	5.9	2.7	3.4

Projet — Compresseur d'images par SVD

Objectif : utiliser la SVD pour compresser une image en niveaux de gris en conservant seulement ses k plus grandes valeurs singulières.

1. Charger une image, la convertir en niveaux de gris, puis la transformer en matrice. Utiliser une image d'environ 256×256 ou 512×512 .
2. Calculer U , Σ et V^T avec `numpy.linalg.svd`.
3. Reconstruire l'image avec $k = 5, 20, 50$ et 100 valeurs singulières.
4. Comparer pour chaque k la qualité visuelle, l'erreur de reconstruction (par exemple MSE) et le nombre de paramètres nécessaires à l'approximation.
5. Produire un graphique de l'erreur de reconstruction en fonction de k , avec $k \in \{5, 20, 50, 100\}$.
6. Livrable : notebook Jupyter contenant code, images originales/reconstruites, mesures et graphique.
7. Publier sur GitHub avec un README expliquant en un paragraphe le principe de la SVD et pourquoi les plus grandes valeurs singulières permettent une approximation.

Contraintes : pour les parties « from-scratch », écrire d'abord le pseudocode puis l'implémentation. Ne pas utiliser une fonction toute faite qui réalise directement l'algorithme demandé.